

Tag object:-

→ human readable permanent pointer to a commit.

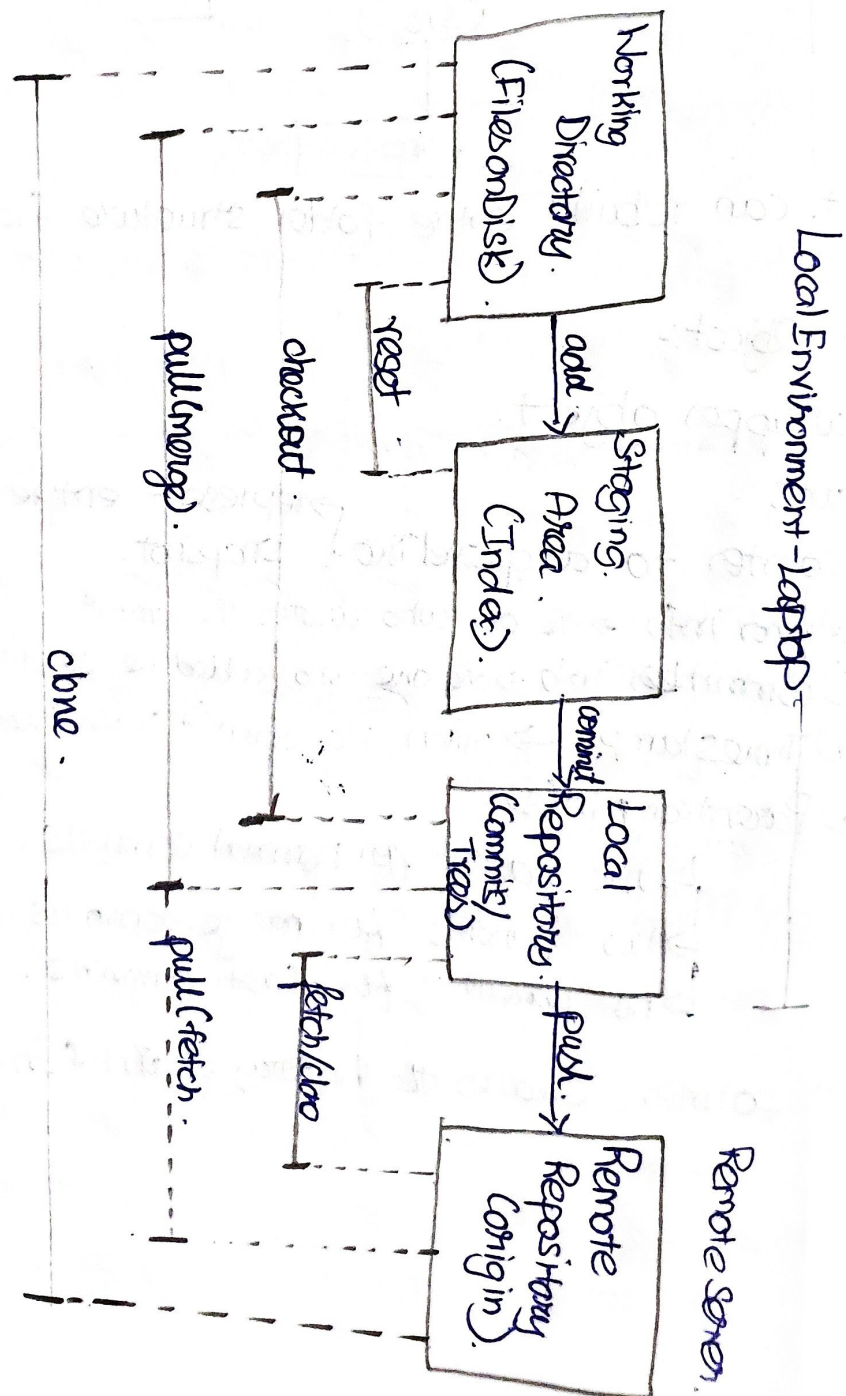
Examples:-

v1.0

v2.0-beta, release-2025.

→ give a name to commit so developers can refer to it later.

Data flow map:-

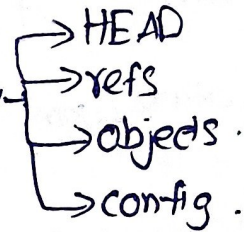


Git Commands:-

1) git init:-

Internal mechanism:-

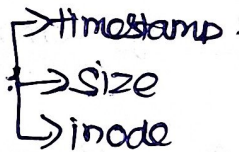
- Involve creating the .git directory.
- The default branch pointing to nothing until the first commit.



2) git status:-

Internal mechanism:-

- Runs stat() system calls on files in the working directory.
- compares file metadata against the .git/index (Staging area).



.git/index (Staging area).

3) git diff:-

Internal mechanism:-

- git diff - compares WD vs Index (Staging area)
- git diff --cached - compares Index vs HEAD.
- git diff HEAD - Working Director vs HEAD.
- git diff --name-only - change in a specific file.
- git diff --stat - summary of changed files.

4) git blame <filename> .

- In that specific file, shows about last commit that modified each line.

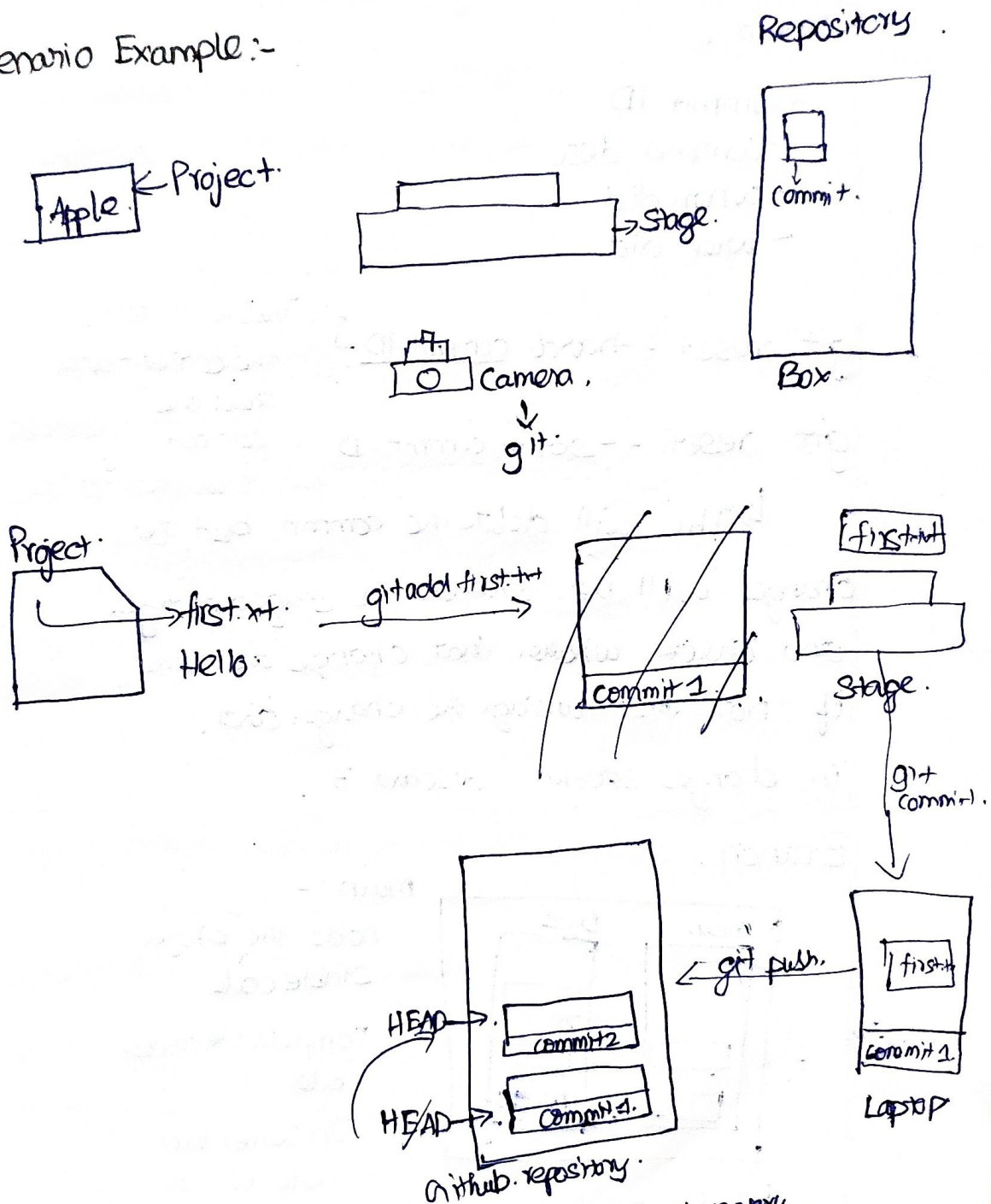
5) git log → Traverses the entire DAG graph from HEAD down to the root.

- displays the history of what happened in that github repo.

Git:-

- ↳ stupid content tracker.
- Acts like a time machine for your files. ↳ backup option.
- allows to keep track of changes made to code or files in projects.

Scenario Example:-



Head pointer. files will be in the local directory.
or code

Branch - subfolder in the repository.

git config commands are used to mention in the commit such that this user only has done this commit activities.

Commit:-

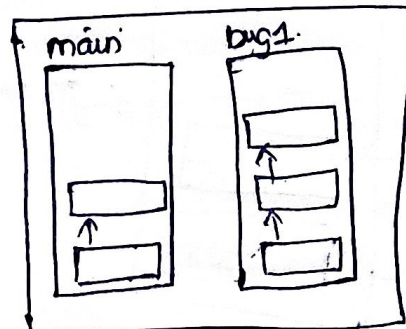
- Commit ID
- Commit desc
- Whom did
- When did.

git reset --hard commit-ID → This will delete the content and also the commit.

git reset --soft commit-ID.

↳ This will delete the commit but the changes will be shown in staged changes and check whether that changes are needed. If not then unstage the changes and in changes section discard it.

Branch.



Repo.

main:-

holds the project stable code

Completed features code

In server this code will be running.

a clone of the main branch is taken and the changes or commits are made upon this branch.

Once the features are completed or either the bug is fixed then the branch is being merged to main.

Ctrl + Shift + P

↳ lists git commands.

main

main (C₁) ← HEAD

main (C₂) ~~← HEAD~~ ← HEAD.

git branch bug-fix.

HEAD → main (C₂) bug-fix.

git checkout bug-fix.

main (C₂) bug-fix ← HEAD.

(C₃) bug-fix ← HEAD.

(C₄) bugfix ← HEAD

git checkout main → git merge bug-fix

HEAD → main (C₂) bug-fix.

(C₃) bug-fix

(C₄) bug-fi.

(M)

In that merging process, using.

fast forward method.

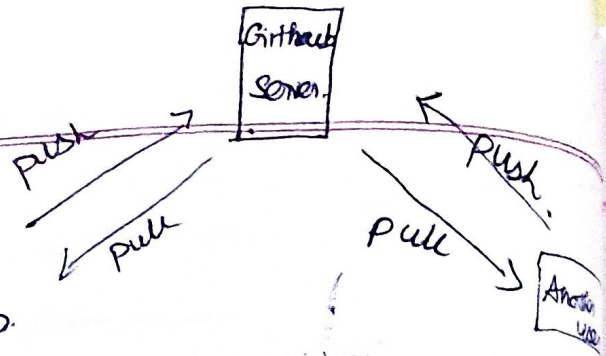
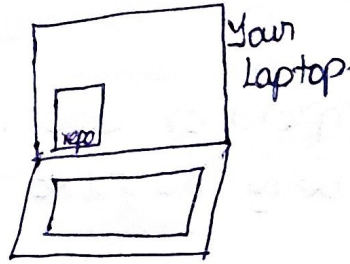
code in bug-fix.

branch is merged to main branch.

bug-fix.

git branch -d bugfix → deletes the ~~other~~ branch, while in other branch (main).

Git Remote:-

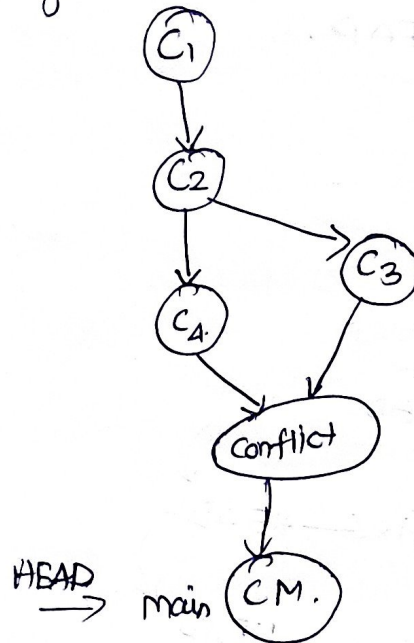


git remote.

git push origin main

git pull origin main

Merge Conflict:-



LLLLLLLL HEAD (main branch)

hello main

=====

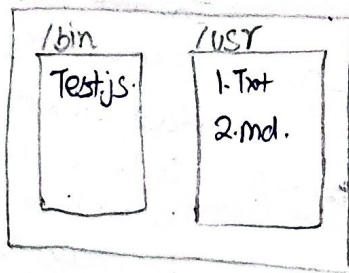
hello feature.

LLLLLLLL Feature.

git commit

Git Internals - Git Objects.

→ maintaining a file-system.



just pure

Blob - contents of a file, no metadata

file - contains content of the file along with the metadata

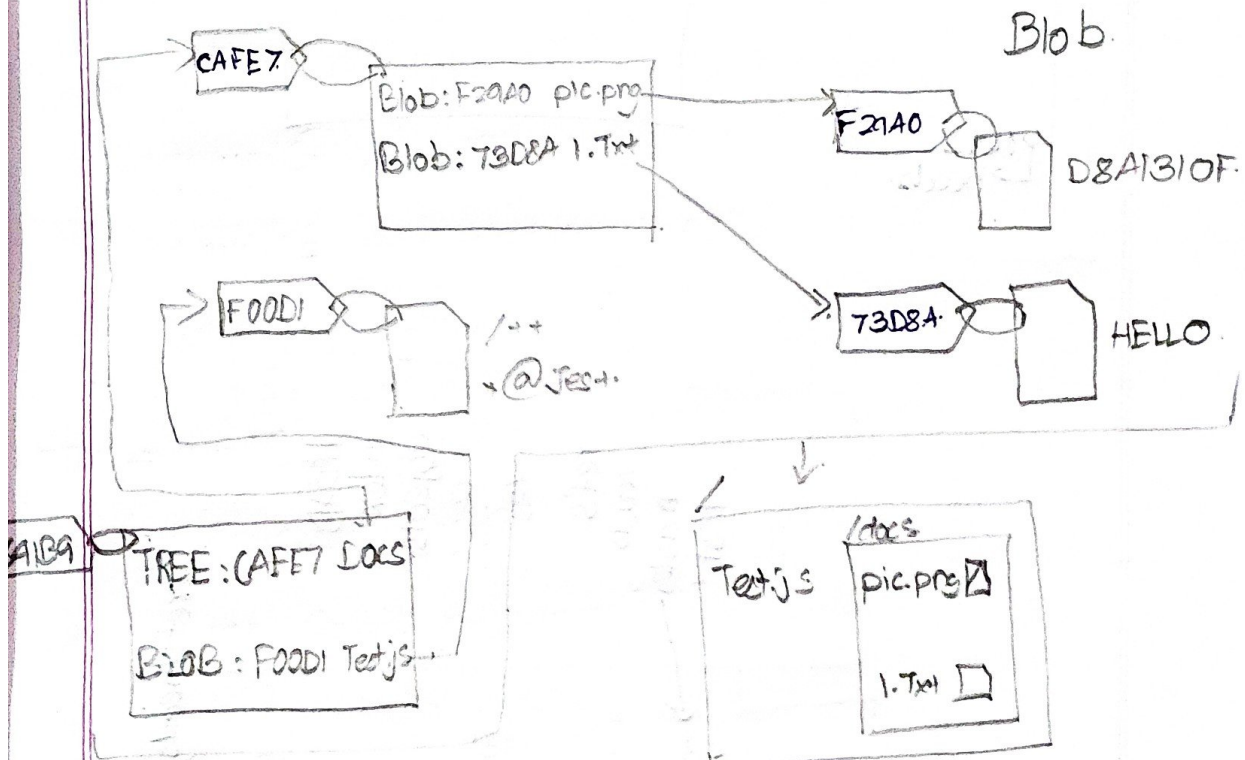
Every blob is identified by sha-1 hash 20 bytes.
40 hex chars.

A directory in normal usage is referred to a tree in git.
Tree

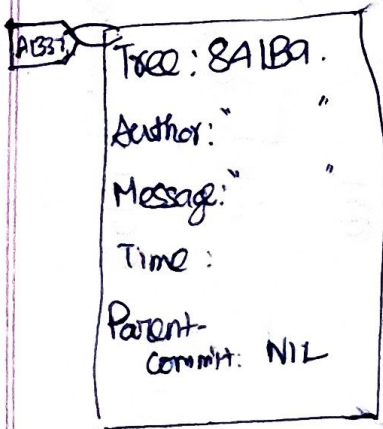
→ A directory & Listing.

→ contains both blobs and trees.

→ identified by sha-1 hash.



Commit - A snapshot of the working tree.



→ Whenever a file's content is being changed, a new blob is created, a new sha-1 hash also being created.

→ Due to cascading effect, the parent's sha-1 hash is being changed.

→ But the ~~new~~ old files (blob) remain same.

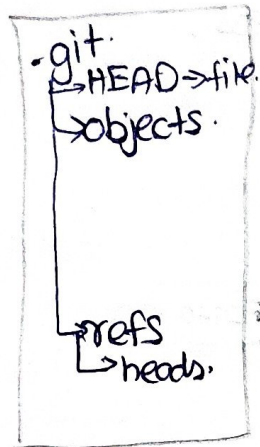
→ So in a new commit, the old references point to old blob itself.

The sha-1 values of blob and tree may be same in two different machines. But the sha-1 value of commit will be different.

• git.

→ A collection of objects.

→ A system for naming those objects



HEAD.

ref: refs/heads/master.

